Home Lab Project Week 9, Day 4 revised

Report: Restrictive Inbound Rules and Default Deny

Objective and summary

Task: configure restrictive inbound firewall rules and establish a default-deny approach for the DMZ, the lab segment that holds the Ubuntu server.

The DMZ is the right segment to lock down because it holds the most exposed machine: a server that accepts connections. Default deny means every inbound connection is blocked unless a rule explicitly allows it.

The resulting policy:

> *"All traffic into the DMZ is **denied by default**, except for explicitly defined traffic destined for specific public-facing services (like HTTP/HTTPS), which is strictly monitored and logged."*

Lab architecture

OPNsense is the only VM on the NAT Network, so Kali and Ubuntu reach each other and the internet only through it.

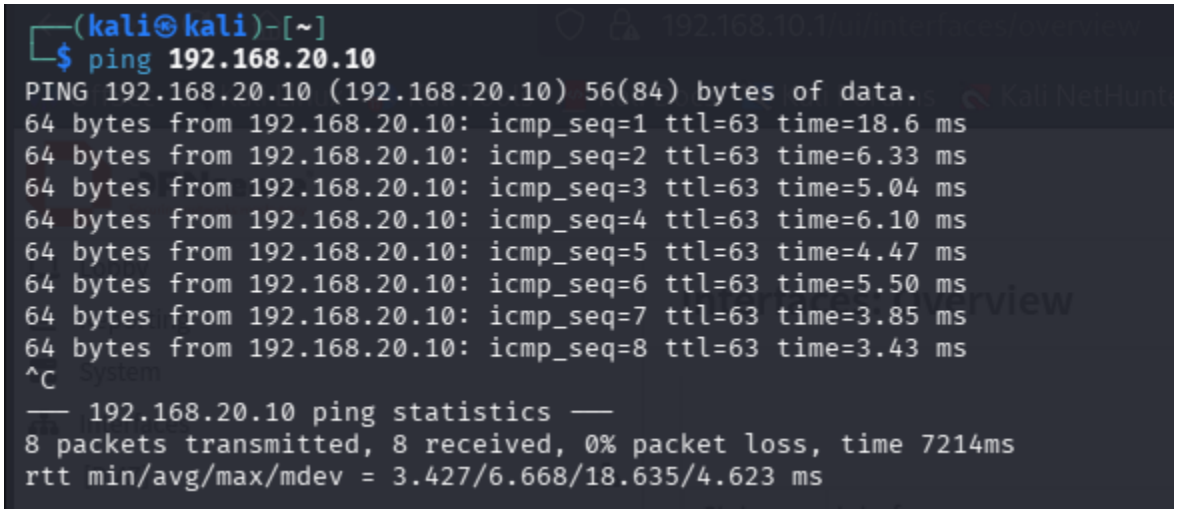
VM	Adapter	VirtualBox network	Interface	IP address	Gateway
OPNsense	1	NAT Network	em0 (WAN)	10.0.2.4/24 (DHCP)	10.0.2.1

OPNsense	2	Internal `intnet`	em1 (LAN)	192.168.10.1/24	none
OPNsense	3	Internal `dmz`	em2 (UbuntuLAN, opt1)	192.168.20.1/24	none
Kali Linux	1	Internal `intnet`	eth0	192.168.10.10/24	192.168.10.1
Ubuntu	1	Internal `dmz`	enp0s3	192.168.20.10/24	192.168.20.1

Starting point from Day 3

Day 3 built the DMZ: a third OPNsense interface (em2) on the VirtualBox internal network `dmz`, addressed 192.168.20.1/24, with the Ubuntu server moved to 192.168.20.10. The full build and its troubleshooting are in the Day 3 report.

At the start of Day 4, the LAN still had OPNsense's default "allow LAN to any" rules, so Kali could reach anything in the DMZ on any port. Today's work replaces that open default with a deny-by-default policy.

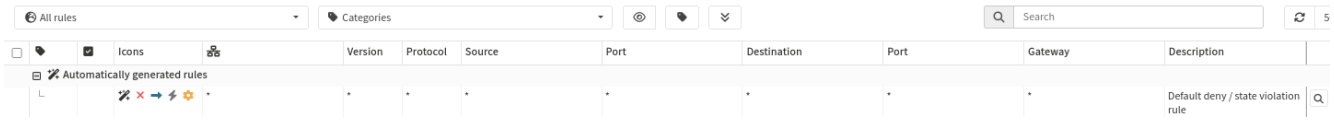


Restrictive inbound rules and default deny

The policy was built in three steps: remove the open default, deny everything into the DMZ, then add the minimum exceptions. In OPNsense, rules apply to traffic entering an interface and the first match wins, so exceptions sit above the deny rule.

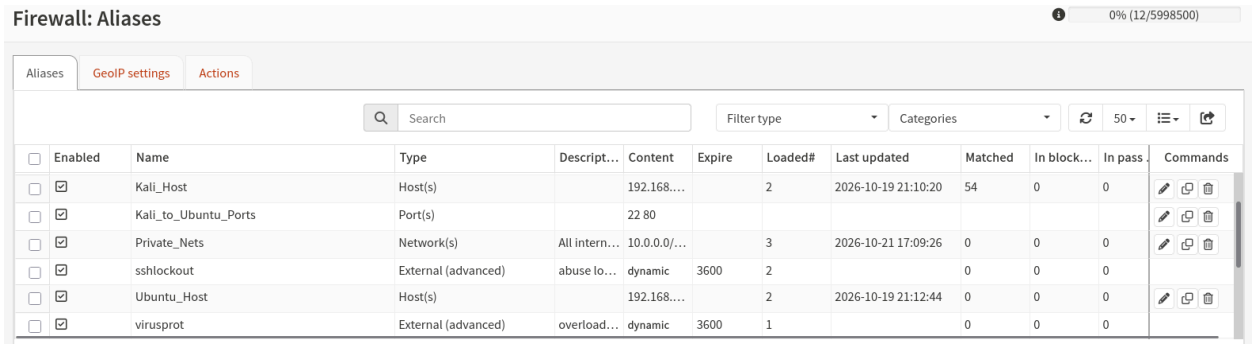
Step 1: Remove the open default (Firewall → Rules → LAN)

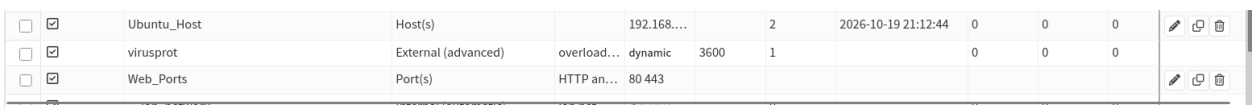
Both "Default allow LAN to any" rules (IPv4 and IPv6) were disabled, not deleted. The anti-lockout rule keeps the web GUI (443 on 192.168.10.1) reachable from the LAN.



Step 2: Aliases for the hosts and services (Firewall → Aliases)

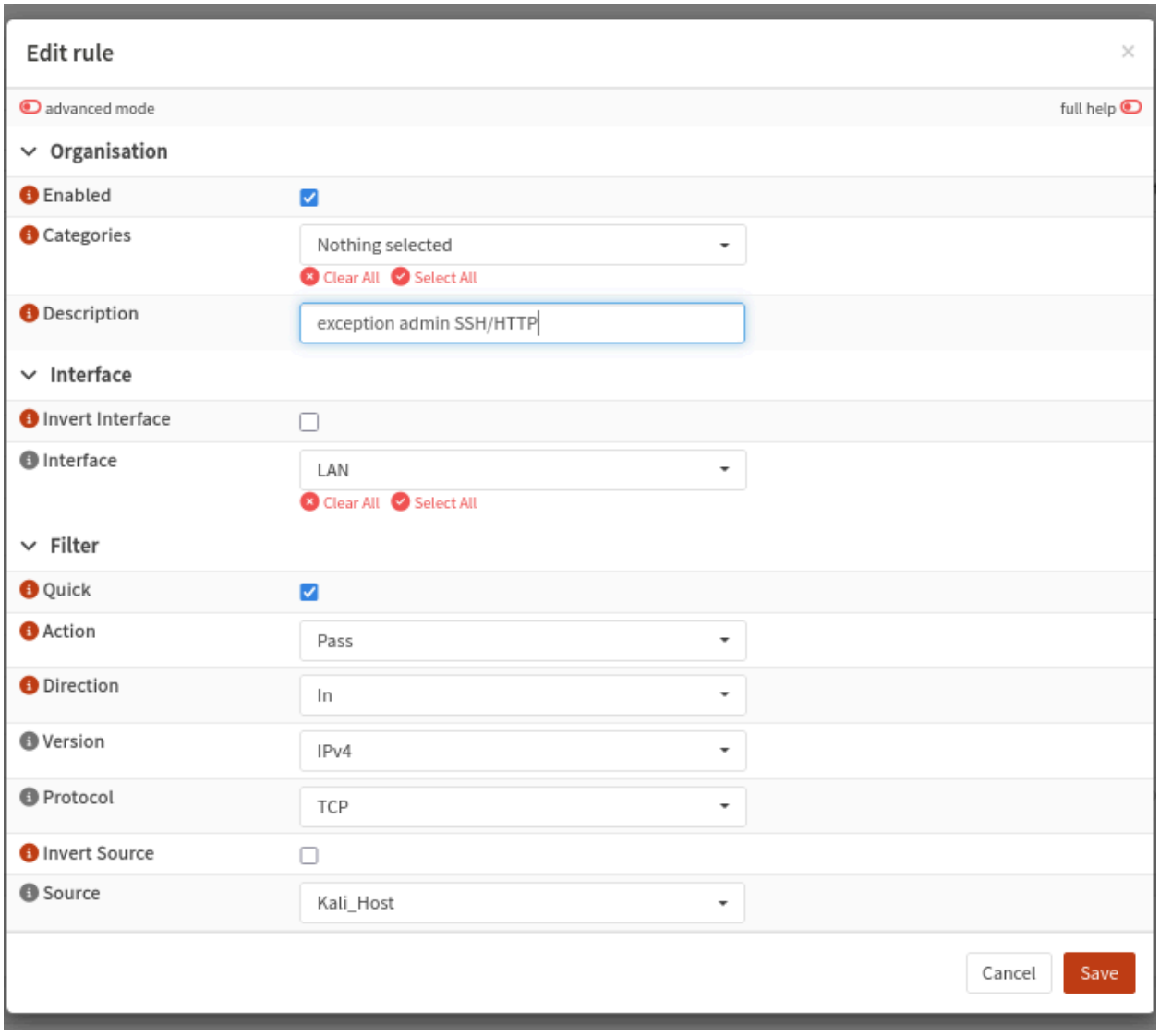
Name	Type	Content	Purpose
KALI_ADMIN	Host(s)	192.168.10.10	Only host allowed into the DMZ
UBUNTU_SRV	Host(s)	192.168.20.10	DMZ server
SRV_ALLOWED_PORTS	Port(s)	22, 80	SSH and HTTP only

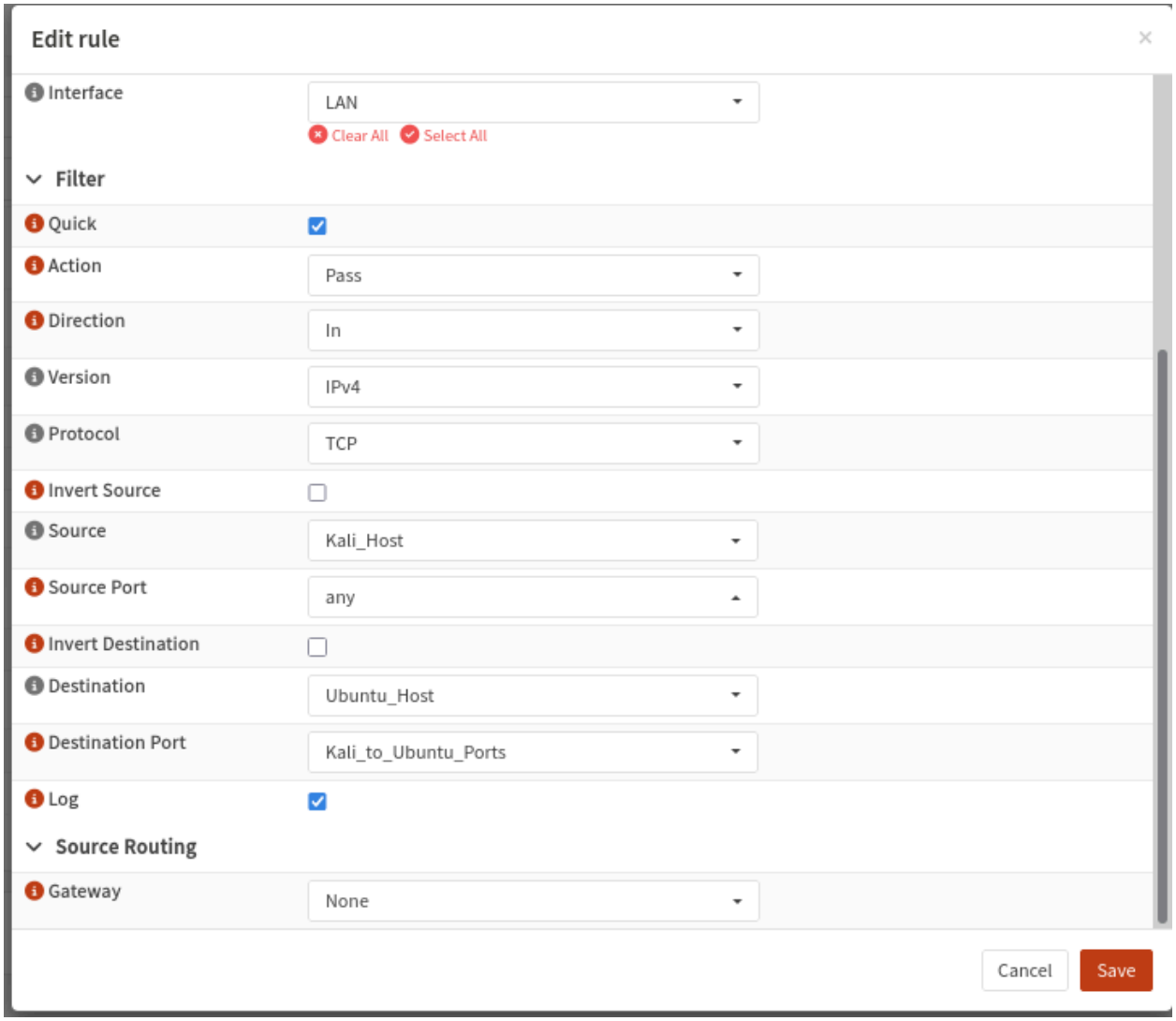


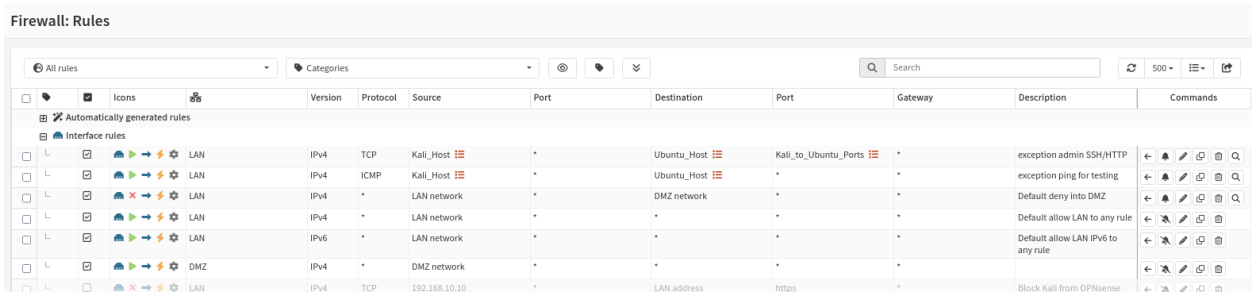


Step 3: Inbound to the DMZ: deny by default, minimal exceptions (Firewall → Rules → LAN)

Order	Action	Protocol	Source	Destination	Port	Log	Description
1	Pass	TCP	KALI_ADMIN	UBUNTU_SRV	SRV_ALLOWED_PORTS	Yes	Exception: admin SSH/HTTP
2	Pass	ICMP echo request	KALI_ADMIN	UBUNTU_SRV	any	Yes	Exception: ping for testing
3	Block	any	LAN net	DMZ net	any	Yes	Default deny into DMZ


Edit rule
advanced mode
full help
Organisation
Enabled
Categories
Nothing selected
Clear All
Select All
Description
exception admin SSH/HTTP
Interface
Invert Interface
Interface
LAN
Clear All
Select All
Filter
Quick
Action
Pass
Direction
In
Version
IPv4
Protocol
TCP
Invert Source
Source
Kali_Host
Cancel
Save





Step 4: Traffic leaving the DMZ (Firewall → Rules → DMZ)

Order	Action	Protocol	Source	Destination	Port	Log	Description
1	Block	any	DMZ net	LAN net	any	Yes	DMZ cannot reach the LAN
2	Pass	TCP/UDP	DMZ net	DMZ address	53	No	DNS to firewall
3	Pass	TCP	DMZ net	any	80, 443	No	Web for updates

Anything not matched on the DMZ interface falls to OPNsense's built-in default deny. Because rule 1 blocks the LAN first, "any" in rule 3 can never reach Kali.

Step 5: WAN baseline

The WAN denies all unsolicited inbound traffic by default. To confirm, check for no pass rules under Firewall → Rules → WAN, an empty Firewall → NAT → Port Forward list, and "Block private networks" and "Block bogon networks" checked on the WAN interface.

Open question: with the LAN default allow disabled, Kali needs its own DNS and web rules on the LAN interface for internet access.

Testing the default deny

Each test checks that only the allowed exceptions get through and everything else is dropped. Fill in the Result column from your own runs.

Prerequisites on Ubuntu: `openssh-server` and `apache2` running, and a decoy service on port 8080 (`python3 -m http.server 8080 &`). The decoy proves the firewall blocks a port that is open on the server.

From	Command	Expected	Proves	Result

Kali	`nmap -Pn -p 22,80,443,8080 192.168.20.10`	22, 80 open; 443, 8080 filtered	Only the exceptions pass
Kali	`nc -zv -w 3 192.168.20.10 8080`	Times out	Default deny drops a running service
Kali	`curl -I http://192.168.20.10`	HTTP/1.1 200 OK	HTTP exception works
Kali	`ping -c 4 192.168.20.10`	Replies	ICMP exception works
Ubuntu	`ping -c 2 -W 2 192.168.10.10`	Fails	DMZ cannot reach the LAN
Ubuntu	`sudo apt update`	Succeeds	DNS and web exceptions work
Ubuntu	`ping -c 2 8.8.8.8`	Fails	Everything else outbound is denied

"Filtered" means the firewall dropped the packet silently; "closed" means the host answered that nothing listens there. Port 8080 showing filtered while the decoy runs is the clearest proof of default deny.

Evidence checklist

- Firewall → Rules → LAN with the default allow rules disabled and the three DMZ rules in order
- Firewall → Rules → DMZ
- Firewall → Aliases

- Firewall → Rules → WAN (no pass rules) and Firewall → NAT → Port Forward (empty)
- Kali terminal: nmap, nc, curl and ping results
- Ubuntu terminal: failed ping to Kali, `apt update`, failed ping to 8.8.8.8
- Firewall → Log Files → Live View showing pass and block entries

Key concepts and reflection

The main lesson: a secure policy starts by denying everything, then opens only what is needed, never the other way around.

Concept	What it means in this lab
Default deny	All traffic into the DMZ is blocked unless a rule allows it
Restrictive inbound	Only SSH, HTTP and ping may enter the DMZ, and only from Kali
Implicit vs explicit deny	OPNsense blocks unmatched traffic on its own; the explicit, logged block rule makes the denials visible as evidence
First match wins	Exceptions sit above the deny rule, or they never trigger
Stateful filtering	Replies to an allowed connection return without a separate rule
"Inbound" in OPNsense	Rules apply to traffic entering an interface, so Kali-to-DMZ rules live on the LAN interface
Least privilege	The server gets only the access it needs: DNS and web for updates, nothing into the LAN

Aliases	Named hosts and ports; changing `UBUNTU_SRV` updates every rule that uses it

Reflection prompts

1. Why is it safer to start from "deny all" and add exceptions than to start from "allow all" and add blocks?
2. Why is the DMZ, rather than the LAN, the right segment for the strictest inbound policy?
3. Port 8080 was running on Ubuntu but showed as filtered from Kali. What does that tell you about network versus host controls?
4. If Ubuntu later needed to send logs to a collector on the LAN, what is the smallest rule change that would allow it?